摘要

本工程由电流互感取能与传感模块、SS16 桥式整流及储能模块、MOS 管分时开关、可编程增益测量模块、MSPM0G3507 控制模块、本地显示与蓝牙通信模块、Android 无线读表器组成。空闲期以取电储能为主，测量窗口由 PA26/PA8 控制 PMOS、NMOS 接入传感前端；控制器完成自动量程和真有效值计算，手机端完成远程读取、报警、记录与趋势显示。

关键词：无源电流表；MSPM0G3507；真有效值；自动量程；低功耗；BLE

一、 系统方案论证与比较

1．取能与传感方案的论证与选择

方案一：使用两组独立次级线圈分别驱动取电电路和传感电路。该方案中两条通道互相隔离，调试较为直观，但两组绕组同时占用磁芯窗口，绕制工作量较大，匝数和耦合差异还会引入一致性误差。

方案二：使用一组带抽头的次级线圈，并以受控 MOS 开关对取电和传感进行分时复用。空闲期断开传感前端，使互感器输出优先经 SS16 全桥向储能电容充电；进入测量窗口后，PA26 拉低使 PMOS 导通、PA8 拉高使 NMOS 导通，传感前端接入并由储能节点维持系统供电。该方案减少重复绕组，且避免传感负载在冷启动阶段持续占用取能功率。

综上所述，本系统采用方案二。次级绕组总计 270 匝并设抽头。经不同抽头组合测试，最终以 170 匝有效绕组完成取电；取电与传感不再作为两个持续并联负载，而由 MOS 开关限定传感支路的接入时间。

2．整流及稳压电源方案的论证与选择

三个方案的前段相同：互感器次级经全桥整流后向储能电容充电，齐纳二极管 D_{Z} 限制储能节点电压。箝位不可省略，线路电流达到 2.2 A 时次级开路电压远高于器件耐压。三者的区别只在储能节点之后靠什么得到 3.3 V，电路如图 1 所示。

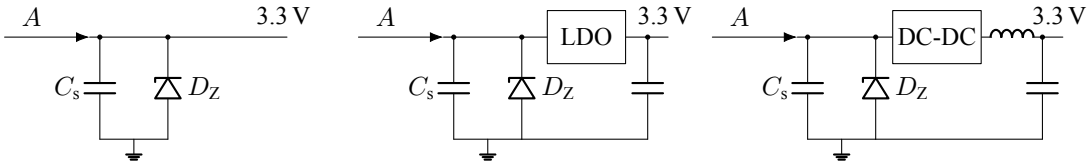
方案一：储能节点直接作为工作电压，不设稳压器。器件最少，静态电流只有齐纳的漏电流。但箝位点即工作电压，储能电容只能充到 3.3 V 附近，而电容储能与端电压平方成正比，同容值下可存能量被压到最小；线路电流升高后多余功率也全部由齐纳耗散。

方案二：箝位点取在远高于 3.3 V 处，储能电容工作在该电压，再由低静态电流 LDO 降到 3.3 V。储能电压提高之后同容值电容存的能量按平方增加，LDO 静态电流可以做到微安级，无电感、无开关纹波。但线性稳压搬运的是电荷而不是能量：输入端流过多少电荷，输出端就只得到多少电荷，储能电容自 V_{H} 放到 V_{L} 期间交付负载的能量为 $3.3\,C_{\mathrm{s}}(V_{\mathrm{H}}-V_{\mathrm{L}})$，其余压差在稳压器上变成热，且储能电压越高浪费的比例越大。

方案三：箝位与储能同方案二，稳压改用降压型开关变换器。变换器按功率守恒工作，同一段放电可交付 $\eta\cdot\frac{1}{2}C_{\mathrm{s}}(V_{\mathrm{H}}^{2}-V_{\mathrm{L}}^{2})$，高压段的能量同样取得出来，储能电压越高优势越大。代价是变换器自身静态电流高于 LDO，需要电感，输

出带开关纹波。

启动电流和启动时间取决于取能功率要用多久才攒够一次测量所需的能量，一次约 260 ms 的测量窗口还要由储能电容独立承担，因此评价稳压方案的标准是同样一只储能电容能交付多少能量。方案一把储能电压钉在 3.3 V，可用能量最小；方案二取不出高压段的能量，且储能电压越高损耗比例越大，与提高箝位点的初衷相抵。综上所述，本系统采用方案三，箝位点取在储能电容和变换器输入耐压以内的较高电平。开关纹波不影响测量精度：ADC 和 DAC 的基准取自芯片内部 2.5 V 基准而非供电电压，纹波不会经基准进入读数。



(a) 箝位直供　(b) 低静态电流 LDO　(c) 降压 DC-DC

图 1 三种稳压方案

3．电流有效值测量及量程方案的论证与选择

方案一：采用外部偏置与外部运算放大电路。由独立基准电路将交流采样信号偏置到 ADC 输入范围中点，再经外部运放进行放大。该方案电路独立，调试直观，但需增加运放、偏置基准及增益切换器件，占用电路板面积，同时会带来额外静态功耗。

方案二：采用 MSPM0G3507 内部 OPA 和 DAC。DAC 产生可调偏置，内部 OPA 完成信号放大，通过软件配置 1、2、4、8、16、32 倍增益，并根据采样幅值自动切换量程。该方案外围器件少、功耗较低，且偏置和增益可由程序协同调整。综合考虑无源取能条件下的功耗和硬件规模，本设计选择方案二，并使用 ADC 采样数据计算真有效值。

二、　理论分析与计算

1．电流互感器与绕组参数分析

电流互感器原边串入被测交流线路，原边匝数满足 $N_1 \leq 5$，副边匝数为 N_2，其结构示意如图 2 所示。

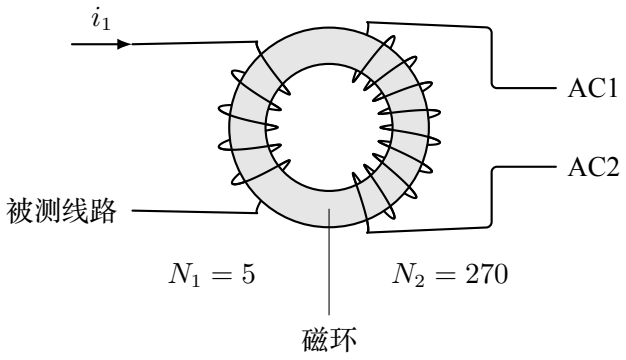


图 2 电流互感器结构示意图

忽略励磁电流与损耗时，原、副边安匝近似平衡，即

$$N_1 I_1 = N_2 I_2, \tag{1}$$

因此副边电流为

$$I_2 = \frac{N_1}{N_2} I_1. \tag{2}$$

增加副边匝数能够提高低电流时的感应电压，但同时会增加绕组电阻和分布参数。为兼顾取能和调试，次级采用 270 匝带抽头结构。试验时分别比较不同有效匝数下的整流电压、带载稳定性和启动能力，最终选取 170 匝有效绕组供电。

2．桥式整流与储能分析

取电支路由四只 SS16 肖特基二极管组成全桥，整流输出记为节点 A，电路如图 3 所示。储能电容不仅承担冷启动蓄能，还要在采集支路接入期间维持 MCU 和模拟前端供电。

节点 A 与储能电容之间另串一只肖特基 D_5，这是分时复用得以成立的前提：采集支路接入时把 A 拉低，若无 D_5，储能电容会经采集支路反向放电，冷启动积攒的能量在第一次测量中即被耗尽。D_5 使能量只能单向流入电容，采集期间系统由电容独立维持。选肖特基同样是为了压低正向压降——低电压下这一项对可用能量的影响最直接。

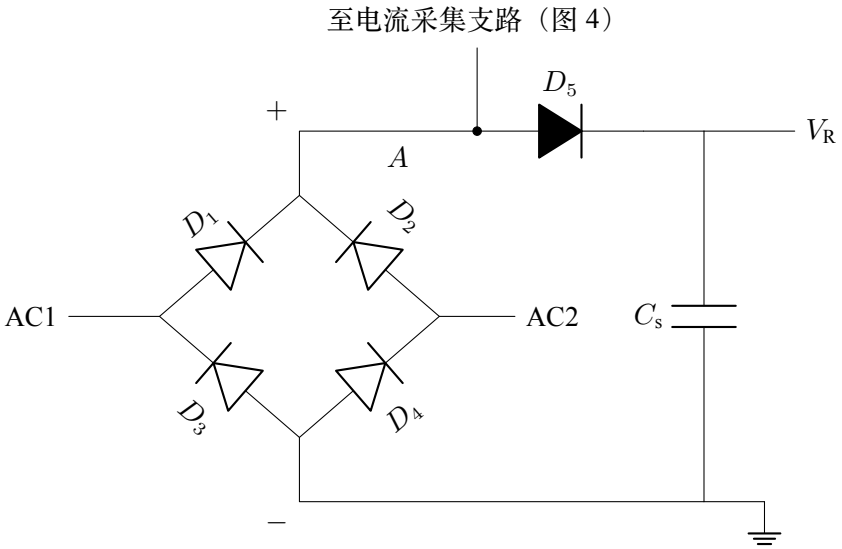


图 3 整流、隔离与储能电路

桥式整流每半周均有两只二极管串联导通。若互感器次级瞬时电压为 $u_2(t)$，单只 SS16 的正向压降为 V_F，则整流后的近似瞬时电压为

$$u_r(t) = \max\left(|u_2(t)| - 2V_F,\, 0\right). \tag{3}$$

由式（3）可知，输入电压越低，二极管压降对可用能量的影响越明显，故选用肖特基桥。储能电容需要同时满足 MCU 完成一次探测、正式采样、计算及外设开启过程中的能量需求。最低启动电流应在电容完全放电的条件下实测，不能仅由空载整流电压判断。

3．取电与传感分时切换分析

采集支路自节点 A 取出，由 PMOS 高边与 NMOS 低边双端把关，采样电阻 R_s 串于两者之间，PMOS 漏极与 R_s 的交点送 PA18 供 ADC 采样，原理如图 4 所示。两只 MOS 同时导通时支路成回路，R_s 上的压降随互感器次级电流变化；任一只关断即整条支路断开，不在测量窗口内不消耗取能功率。

PA26 控制 PMOS 栅极并由 10 kΩ 电阻上拉至 A，PA8 控制 NMOS 栅极并由 10 kΩ 电阻下拉至地。上电至 MCU 接管 GPIO 之前引脚是输入而非驱动输出，两只电阻在这段窗口内使支路保持断开——而这段窗口恰好是储能电压最弱、最经不起额外负载的时候。

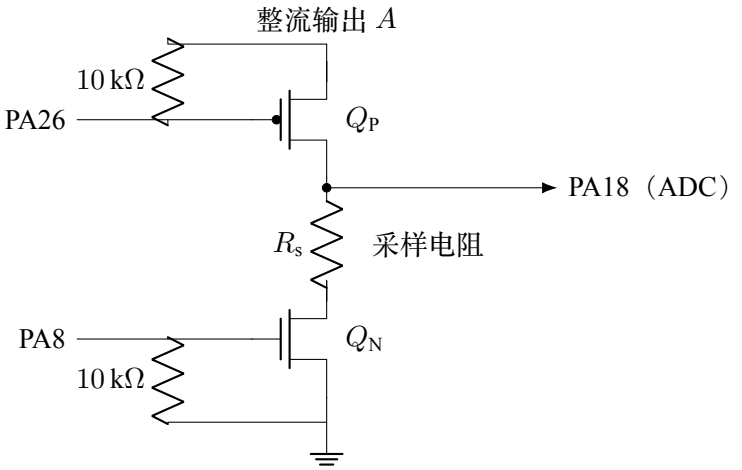


图 4 电流采集支路与双端 MOS 开关

表 1 取电与传感分时控制状态

工作状态	PA26	PA8	MOS 状态	能量与信号路径
空闲/取电	高	低	PMOS、NMOS 均关断	传感前端隔离，互感器输出优先用于整流储能
测量/传感	低	高	PMOS、NMOS 均导通	传感前端接入，储能节点维持 MCU 和采样电路供电

测量触发后先切换为传感态，并预留 20 ms 建立时间；随后进行 40 ms 探测采样、最多 1 ms 换档建立及 200 ms 正式采样。测量结束后先关闭 TIMG6/ADC1，再关闭 OPA1/DAC12，最后恢复 PA26 高、PA8 低。若分时窗口内储能电压从 V_{H} 下降到 V_{L}，则至少应满足

$$\frac{1}{2}C_{\mathrm{s}}\left(V_{\mathrm{H}}^2-V_{\mathrm{L}}^2\right)\geq\frac{E_{\mathrm{measure}}}{\eta},\tag{4}$$

其中 E_{measure} 为一次约 260 ~ 261 ms 测量窗口的能耗，η 为稳压链路效率。该条件由 VRECT 与 3.3 V 波形实测验证。

4．可编程增益与偏置计算

对增益为 G 的同相偏置放大电路，其输出可表示为

$$V_{\mathrm{o}}=GV_{\mathrm{i}}+(1-G)V_{\mathrm{DAC}}.\tag{5}$$

若输入直流中心为 $V_{\mathrm{i,dc}}$，希望输出中心位于 ADC 中点 V_{c}，则 $G>1$ 时的 DAC 电压为

$$V_{\mathrm{DAC}}=\frac{GV_{\mathrm{i,dc}}-V_{\mathrm{c}}}{G-1}.\tag{6}$$

固件以 12 位 ADC 的 2047 码为目标中心，探测阶段采集 160 点，时间为 40 ms；正式阶段采集 800 点，时间为 200 ms。目标峰峰值利用率设为满量程的 65%，回差下、上限分别取 25% 和 75%。

5．真有效值及标定算法

定时器 TIMG6 以 4 kHz 产生 ADC1 转换事件，DMA 将数据搬运至内存。对 N 点采样序列 $x[n]$ 使用 Hann 权重 $w[n]$，其加权均值为

$$\bar{x}_w = \frac{\sum_{n=0}^{N-1} w[n]x[n]}{\sum_{n=0}^{N-1} w[n]}. \tag{7}$$

去除直流分量后的有效值码宽为

$$X_{\mathrm{rms}} = \sqrt{\frac{\sum_{n=0}^{N-1} w[n]\,(x[n]-\bar{x}_w)^2}{\sum_{n=0}^{N-1} w[n]}}. \tag{8}$$

Hann 权重可降低采样记录没有覆盖整数工频周期时的边界影响。各量程分别保存标定点 (x_k, I_k)。当 $x_k \le X_{\mathrm{rms}} \le x_{k+1}$ 时，采用

$$I = I_k + \frac{X_{\mathrm{rms}} - x_k}{x_{k+1} - x_k}(I_{k+1} - I_k) \tag{9}$$

进行分段线性插值。六档各有一张标定表；表中数据由台架比对写入，名义比例系数只可用于调试，不能作为 0.5% 精度的证明。

三、 电路与程序设计

1．系统框图

系统由电流互感器、SS16 桥式整流、储能与稳压、PMOS/NMOS 分时开关、可编程模拟前端、MSPM0G3507、OLED、受控供电的 HC-42 以及 Android 读表器组成。电流表 A、B 结构相同且供能与测量链路相互独立，拆除任意一表不会影响另一表。硬件系统框图如图 5 所示。

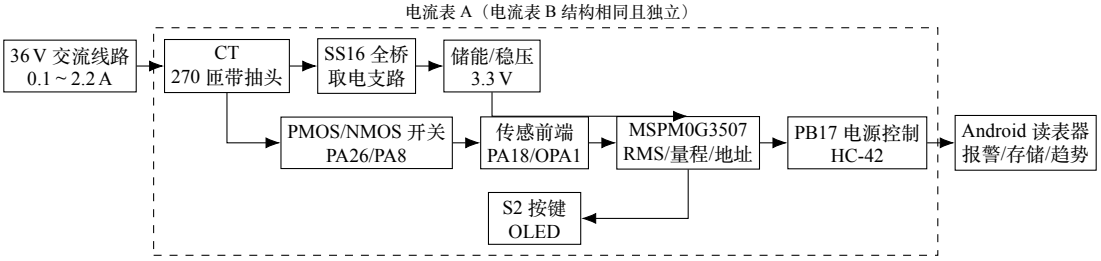


图 5 硬件系统框图

2．取能、整流与稳压电路设计

取能绕组采用 270 匝总绕组并设置抽头，最终选用其中 170 匝有效绕组为整流桥供电。四只 SS16 组成全桥，使正、负半周均向储能电容充电。空闲期传感前端被双端 MOS 开关隔离，储能节点以较小负载积累能量；测量期则由储能电容承担约 260 ~ 261 ms 的采样脉冲负载。齐纳箝位之后由降压型开关变换器稳压到 3.3 V，方案比较见 1.2。储能电容并接放电通路，便于每次启动试验前恢复相同初始状态。

3．模拟开关、模拟前端与控制器接口设计

PA26 驱动 PMOS 高边开关，栅极上拉后默认关断；PA8 驱动 NMOS 低边开关，栅极下拉后默认关断。测量时 PA26=0、PA8=1，两只 MOS 同时导通；测量结束恢复 PA26=1、PA8=0。两控制脚从不作为独立功能使用。模拟前端提供直通 X1 及 OPA 可编程增益 X2、X4、X8、X16、X32 六档，DAC 根据探测阶段的直流中心调整偏置。MSPM0G3507 的 ADC、OPA、OLED、按键、蓝牙电源、UART 和地址接口分配见附录表 5。

4．电流测量程序设计

程序上电后先把 PA26/PA8 置为取电空闲态，同时将 PB17 拉低并保持 OLED 未初始化。每次触发时先接入传感前端，等待 20 ms 后采集 160 点探测记录，根据峰峰值和直流中心选择增益及 DAC，再采集 800 点正式记录。完成采样后按 TIMG6/ADC1、OPA1/DAC12、外部 MOS 开关的顺序关断，再计算真有效值并查表。程序流程如图 6 所示。

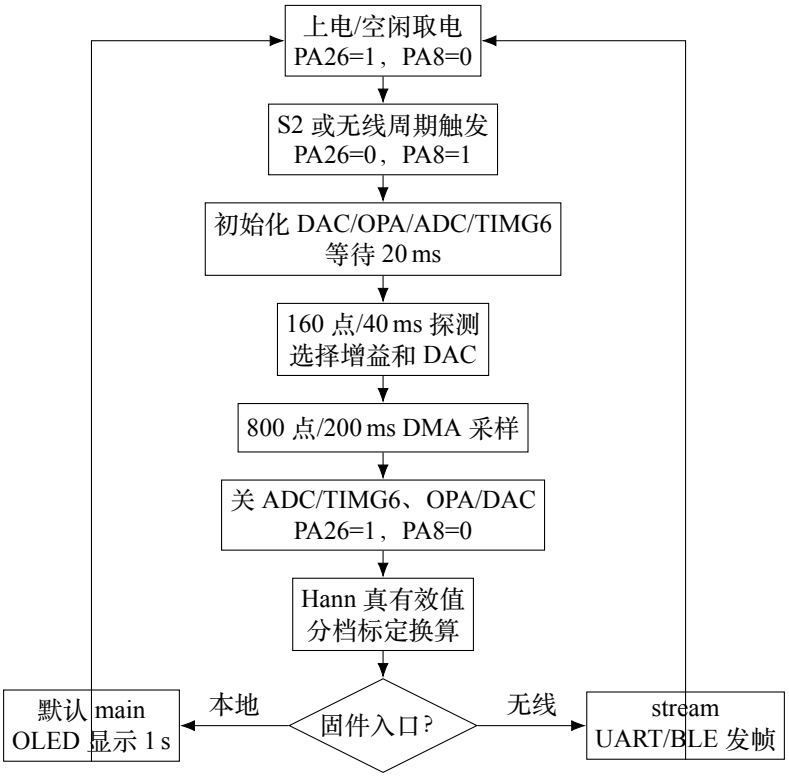


图 6 电流测量与外设控制流程图

5．OLED 与蓝牙电源控制设计

OLED 上电时不发送初始化命令。一次测量由两种请求触发：S2 下降沿，或蓝牙收到 MEAS。两者完全等价、走同一条路径，测量期间再次到达的请求两者都丢弃而不排队。上电时另有一次不经请求的测量，使读表器一连上就有数可读。结果显示 1 s 后熄屏。OLED 对比度命令设置为 0x81,0x0F，空闲期不刷新屏幕。

蓝牙模块不是仅延后 UART，而是由 PB17 控制外部 3.3 V 负载开关。PB17 低电平时负载开关关闭，高电平时导通。无线固件先在蓝牙断电状态完成第一次测量，再将 PB17 置高并等待 350 ms，之后才初始化 UART2。蓝牙断电期间 UART 引脚保持未初始化，避免串口反向灌电。UART2 收发均需初始化：读数由读表器请求产生，请求经 RX 进入。仅计算固件固定时序，蓝牙最早约在 $20+40+200+350=610$ ms 后具备通信条件；若发生换档还需增加约 1 ms，实际启动时间还包括互感取能和储能电压建立时间。

6．无线通信与 Android 读表器设计

题目 2(2) 规定基本模式与自动模式下均不得操作电流表，因此读数由读表器请求产生，电流表不主动上报测量值。链路为 HC-42 透传串口（FFE0 服务、

FFE1 通知特征），速率 115200 bit/s。透传链路不提供分包与长度字段，故采用面向行的 ASCII 协议，以换行作自同步定界符：接收端在任意位置接入或丢失字节后，都能在下一个换行重新对齐。

读表器发出 MEAS,ADDR=n，仅编码开关为 n 的电流表应答。命令带地址而非仅用广播，是因为一台电流表在不同时刻由编码开关赋予不同地址，仅凭连接无法确定对端身份；带地址后请求与应答一一对应，不会把某地址的应答记到另一地址名下。

电流表空闲时每 2 s 播发一次 IMHERE,ADDR=n 保活帧。该帧不含读数，也不唤醒任何模拟电路，其作用有二：编码开关改变时旧地址的心跳停止、新地址出现，读表器据此更新在场地址；负载断开后电流表随之失电，心跳消失，即为题目 2(3) 的离线判据，较等待一次请求超时更快。请求超时只说明本次未读到，不据此判定离线——正在心跳的电流表只是本次未及应答，并非不在。

Android 应用扫描 8 s、间歇 12 s，连接名称符合 HC-42、HC42、METER、AMMETER 或 BYJX_ 前缀的设备。基本模式为一对一手动读取；自动模式一键启动，每 120 s 对当前在场的地址依次请求，同一连接上同时只允许一个未完成请求。读数写入 SQLite，容量 1000 条，历史与趋势均超过题目要求的 30 条；离线与请求超时都不写入记录——记录用于保存读数，缺席不是读数。低于 0.2 A 与高于 2.0 A 的每一次读数都给出声音与振动提示，不以状态跳变为条件：读数稀疏，同一状态的相邻两次读数是两个独立事件。

四、　测试方案与测试结果

1．系统测试方案

（1）测试仪器

测试仪器如表 2 所示。

表 2 测试仪器列表

序号	仪器类别	仪器型号	数量	性能参数及用途
1	交流电源	熙硕 STG-500W	1	提供可调交流测试电源
2	五位半万用表	SIGLENT SDM3055X-E	1	测量标准电流及电压
3	数字示波器	SIGLENT SDS2102X PLUS	1	观察储能、采样及启动时序
4	Android 手机	通用	1	无线读取、报警和数据存储测试
5	秒表	通用	1	记录系统启动及通信时间

（2）测试方法

1、进行电流测量精度测试。

（1）保持线路电压为 36 V，依次设置 0.10、0.20、0.50、1.00、1.50 和 2.00 A，每点稳定后记录标准表与被测表读数。

（2）每个电流点重复测试不少于 5 次，在量程重叠区增加测试点，并分别完成 A、B 两表测试。

（3）拆除电流表 B 后重复测试 A，比较误差变化，验证两表互不影响。

2、进行取能与启动性能测试。

（1）储能电容充分放电后，从较小电流逐步增加，记录能够稳定完成测量和发送的最低电流。

（2）在代表电流点捕获储能电压建立、首次有效值生成和首帧到达时刻。

（3）比较不同抽头组合的整流电压和带载稳定性，记录最终供电匝数。

（4）同时观察 PA26、PA8、VRECT 和 3.3 V：确认空闲态为 PA26=1、PA8=0，测量态为 PA26=0、PA8=1，并记录约 260 ~ 261 ms 传感窗口内的储能电压跌落。

3、进行本地显示和无线功能测试。

（1）确认上电阶段 OLED 无驱动；单击 S2 后完成一次测量，显示 1 s 后自动熄屏。

（2）手机与电流表间距设为 5 m，分别检查人工读取和 120 s 周期搜索。

（3）设置低限、正常、超限并中断一只表通信，检查报警；写入不少于 30 条记录，重启应用后检查历史和趋势数据。

2．测试结果与分析

（1）供电绕组测试

次级绕组总计 270 匝并设抽头。通过不同抽头组合对比整流输出和带载稳定性后，最终选取 170 匝有效绕组供电。该结果已经用于当前取能方案。

表 3 供电绕组抽头对比

阶段	绕组结构	结果
85 匝	单段次级绕组	感应电压和调节余量有限
改进版	270 匝总绕组，每 50 匝预留抽头	可组合不同有效匝数进行测试
最终方案	170 匝有效绕组供电	经抽头对比测试确定

（2）电流测量精度测试

被测显示值 I_{x} 相对标准值 I_{ref} 的相对误差为

$$\delta = \frac{I_{\mathrm{x}} - I_{\mathrm{ref}}}{I_{\mathrm{ref}}} \times 100\%. \tag{10}$$

题目要求 0.1 ~ 2.0 A 全量程内满足 $|\delta| \leq 0.5\%$。表 4 按标准表比对结果逐格填写，不填入推测数据。

表 4 电流测量精度记录表

标准值/A	0.10	0.20	0.50	1.00	1.50	2.00
A 表示值/A	待实测	待实测	待实测	待实测	待实测	待实测
A 表误差/%	待实测	待实测	待实测	待实测	待实测	待实测
B 表示值/A	待实测	待实测	待实测	待实测	待实测	待实测
B 表误差/%	待实测	待实测	待实测	待实测	待实测	待实测

（3）标定表覆盖范围

X1 直采档写入 7 个实测标定点，覆盖 0.0755 ~ 2.030 A，相邻点之间分段线性插值；X2、X4、X8、X16、X32 五档表由实物标定填入。X1 点中 0.191 ~ 0.530 A 间隔较大，2.03 A 以上由末段外推，全量程 0.5% 需独立验收。标定数据由 BLE 与 SDM3055X-E 同步采集工具生成，标准电流、RMS、量程和质量标志逐行写入 CSV。

五、 总结

本系统完成了“无源”交流电流表及无线读表器的总体方案、测量程序和手机端软件设计。取能与传感共用带抽头的次级绕组，抽头对比测试最终确定以 170 匝有效绕组供电。PA26 控制 PMOS 高边、PA8 控制 NMOS 低边，传感

前端只在约 260～261 ms 测量窗口接入；空闲期双管关断，互感器输出优先用于 SS16 整流储能。模拟模块和外部开关按顺序掉电，配合六档自动量程、DAC 偏置、DMA 和 Hann 加权真有效值算法降低无效功耗。OLED 采用一按一测并延后初始化，蓝牙采用 PB17 控制外部电源。Android 读表器能够完成地址识别、人工读取、周期搜索、异常报警、历史记录和趋势显示。

硬件方面还需在储能电容完全放电的初始条件下实测最低启动电流和启动时间，并确认测量窗口的脉冲负载下变换器输出不跌破工作门限。还应实测 PMOS、NMOS 的导通压降、关断漏电及切换瞬态，确认传感窗口内 VRECT 和 3.3 V 不低于系统工作门限，并避免 MOS 体二极管或输入保护网络形成反向供电路径。

标定方面，X1 在 0.1～0.3 A 区间和 2.0～2.2 A 补点，X2～X32 五档各建立独立标定表。系统联调同时接入 A、B 两表，逐项验证地址设置、人工读取、120 s 周期搜索、低限/超限/离线报警、30 条以上记录保存及 5 m 通信距离。

附录

1．控制器接口

表 5 MSPM0G3507 主要接口分配

引脚	功能	说明
PA26	PMOS 高边控制	低电平导通；10 kΩ 上拉，上电默认关断
PA8	NMOS 低边控制	高电平导通；10 kΩ 下拉，上电默认关断
PA18	ADC1 输入	电流采样原始通道
PA16	模拟输入	OPA 可编程增益输出通道
PB2、PB3	软件 I^2C	OLED 的 SCL、SDA
PB21	S2 按键	下降沿触发一次测量，与蓝牙 MEAS 等价
PB17	蓝牙电源使能	高电平导通外部负载开关，低电平关闭
PB15	UART2 TX	HC-42 串口发送，115200 bit/s
PB16	UART2 RX	接收读表器的 MEAS 请求
PB0、PB6～PB8	地址输入	4 位低有效地址，范围 00～15

2．通信协议

读表器 → 电流表：

MEAS　或　MEAS,ADDR=dd

前者为广播，后者仅地址匹配的电流表应答。命令刻意不与上报帧同前缀：两个方向共用同一条透传链路，若电流表认得出自己输出的语法，一次回环即可自触发。

电流表 → 读表器，读数帧：

METER_TEST,ADDR=dd,CURRENT_MA=iiii,STATUS=ssss\r\n

（1）ADDR：十进制两位电表地址，00～15，取自 4 位编码开关。

（2）CURRENT_MA：电流有效值，单位为 mA。

（3）STATUS：NORMAL、LOW 或 HIGH，按 0.2 A 与 2.0 A 判定。此处没有 OFFLINE——离线是读表器根据心跳消失作出的判断，电流表无法报告自己不在。

电流表 → 读表器，保活帧与调试帧：

IMHERE,ADDR=15

METER_CAL,ADDR=15,RMS=193.36,GAIN=1,FLAG=OK,MEAN=395,PP=662

IMHERE 每 2 s 一次，只声明在场。METER_CAL 与读数帧同次产生，其中 RMS、GAIN 用于建立分档标定表，MEAN、PP 为探测帧测得的输入直流与峰峰值，FLAG 为本次读数的质量标志。

读数不可信时（基准未就绪、输入越出转换窗口、采集帧未完成），电流表只发 METER_CAL 并在 FLAG 中给出原因，不发读数帧，避免把不可信的数值以正常状态送出。读表器据此把” 表故障” 与” 表离线” 区分开——二者若不加区分，在读表器看来完全一样。

3．主要程序代码

电流表固件用 Rust 编写，编译目标为 thumbv6m-none-eabi，异步执行器为 embassy。以下按一次测量经过的顺序列出四个源文件：main.rs 决定何时测量，meter.rs 执行一次测量的完整时序，dsp.rs 把采样序列算成有效值码宽，link.rs 管地址、帧格式和收发。其余模块不在此列出：sampler.rs 配置 TIMG6/ADC1/DMA，range.rs 配置 OPA1 增益与 DAC 偏置并实现换档判据，cal.rs 存分档标定表，vref.rs 开关内部基准，supply.rs 用 ADC0 的内部通道读供电电压，oled.rs 与 ui.rs 驱动显示。行号即源文件行号。

代码 1 src/main.rs：触发与调度

```
#![no_std]
#![no_main]

//! "Passive" AC ammeter, circuit A: one reading per request.
//!
//! A request is either a press of S2 or a `MEAS` command off the radio; the two
//! are the same trigger and take the same path. One request runs one
//! `meter::measure`, which switches the external circuit in, takes the reading
//! and puts everything back to sleep. The result goes out over the radio and
//! onto the panel, which stays lit for `DISPLAY_ON_MS` and then goes dark.
//!
//! Power-on takes one reading unasked, before the radio is up, and reports it
//! the moment the port opens. So the meter always has a number to give, and a
//! reader that connects before anyone has pressed anything is not told the
//! meter is silent when it is merely idle.
//!
//! Requests that land inside a cycle are dropped, not queued, from either
//! source: `wait_for_falling_edge` clears pending edges before it arms, and
//! `Link::discard_pending` throws away buffered command bytes at the same
//! point. A reading answers the request that started it.
//!
//! Between cycles nothing runs -- the executor idles on the two triggers, with
//! every analog block powered down and the radio up.
//!
//! This file is the trigger and nothing else. The substance lives in the
//! modules below.
//!   `meter`   -- one reading: AFE control, bring-up, probe, frame, power-down
//!   `link`    -- HC-42 on UART2: the address, the frames out, the command in
//!   `sampler` -- TIMG6/ADC1/DMA and the sample buffers
//!   `range`   -- OPA1 PGA, DAC bias pivot, and the autoranger
//!   `dsp`     -- windowing, true RMS
//!   `cal`     -- per-range LSB -> amps tables
//!   `supply`  -- the rail itself, off ADC0's internal monitor
//!   `ui`      -- the OLED readout, held dark until that rail is up
//!   `led`     -- the power-on indicator

use embassy_executor::Spawner;
use embassy_futures::select::{Either3, select3};
use embassy_mspm0::dma::{self, Channel};
use embassy_mspm0::gpio::{Input, Level, Output, Pull};
use embassy_mspm0::uart::BufferedInterruptHandler;
use embassy_mspm0::{bind_interrupts, peripherals};
use embassy_time::Timer;
use panic_halt as _;

mod cal;
mod dac;
mod dsp;
mod led;
```

```rust
mod link;
mod meter;
mod oled;
mod range;
mod sampler;
mod supply;
mod ui;
mod vref;

use link::{Address, HEARTBEAT_MS, Link};
use meter::Meter;
use ui::{DISPLAY_ON_MS, Panel};

bind_interrupts!(struct Irqs {
    DMA => dma::InterruptHandler<peripherals::DMA_CH0>;
    UART2 => BufferedInterruptHandler<peripherals::UART2>;
});

/// Radio buffers. TX only has to hold the two frames of one reading; RX only
/// has to hold one command line, and anything past that is a reader talking
/// over itself. Both are `static` because `BufferedUart` keeps them for as long
/// as the port is open, which here is forever.
static mut TX_BUF: [u8; 192] = [0; 192];

static mut RX_BUF: [u8; 64] = [0; 64];

/// Idle until something asks for a reading, announcing this meter's address
/// every `HEARTBEAT_MS` in the meantime.
///
/// The beat is not a trigger -- it goes out and the wait resumes. Only the
/// button and a `MEAS` command return from here, and which one did is not worth
/// distinguishing: they are the same request.
async fn wait_for_request(trigger: &mut Input<'static>, link: &mut Link, address:
↪ &Address) {
    loop {
        let addr = address.read();
        match select3(
            trigger.wait_for_falling_edge(),
            link.wait_command(addr),
            Timer::after_millis(HEARTBEAT_MS),
        )
        .await
        {
            Either3::First(()) | Either3::Second(_) => return,
            // Re-read the switch on every beat rather than caching it: moving
            // the switch is the one operation 2(2) allows during a run, so the
            // address has to be able to change between two beats.
            Either3::Third(()) => link.send_alive(address.read()),
        }
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) -> ! {
    let p = embassy_mspm0::init(Default::default());

    // First thing after the clocks, so the indicator is up before anything
    // that can fail or block: from here on, a dark LED means the firmware
    // never got this far.
    //
    // The task pool is one deep and this is its only spawn, so the token
    // cannot come back `Err`. Handled rather than unwrapped anyway: an
    // indicator that failed to start is a reason to run without it, not a
    // reason to halt a meter that is otherwise fine.
    if let Ok(token) = led::blink(Output::new(p.PA0, Level::Low)) {
        spawner.spawn(token);
    }

    let mut meter = Meter::new(p.PA8, p.PA26);
```

```
    let mut dma = Channel::new(p.DMA_CH0, Irqs);
    let mut panel = Panel::new();
    let address = Address::new(p.PB0, p.PB6, p.PB7, p.PB8);

    // LaunchPad S2 is the independent user button on PB21. It pulls the pin
    // low when pressed; unlike S1/PA18, it does not disturb the ADC input.
    let mut trigger = Input::new(p.PB21, Pull::Up);

    // One reading on power-up, before anything can ask for it, and before the
    // radio exists. Measuring first is what the board notes ask for on a
    // harvested rail (6.1): the module's start-up current is the largest single
    // draw in this firmware, and it should not land while the supply has just
    // come up. Nothing is lost by waiting -- the reading is already in hand
    // when the port opens, so the first frame goes out as soon as there is
    // anything to send it over.
    let mut in_hand = Some(meter.measure(&mut dma).await);

    // SAFETY: this is the only place either buffer is named, and the `Link`
    // built from them lives for the rest of the program.
    let mut link = Link::power_up(
        p.PB17,
        p.UART2,
        p.PB15,
        p.PB16,
        Irqs,
        unsafe { &mut *core::ptr::addr_of_mut!(TX_BUF) },
        unsafe { &mut *core::ptr::addr_of_mut!(RX_BUF) },
    )
    .await;

    loop {
        // The power-on reading is already taken; every later one has to be
        // asked for. Both then travel the identical path below, so the first
        // reading is reported exactly like the ones that follow.
        let reading = match in_hand.take() {
            Some(reading) => reading,
            None => {
                // Idle here on both triggers at once. Whichever arrives first
                // wins and the other future is dropped -- for the button that
                // discards a pending edge, and for the radio it leaves any
                // buffered bytes where they are, which the discard below then
                // clears. Which one won does not matter: the two are the same
                // request.
                match &mut link {
                    Some(link) => wait_for_request(&mut trigger, link, &address).
↪ await,
                    // A radio that failed to open leaves the meter working off
                    // its button and its panel, which is strictly better than
                    // refusing to measure at all.
                    None => trigger.wait_for_falling_edge().await,
                }
                meter.measure(&mut dma).await
            }
        };

        // Report before the panel: the reader is waiting on an answer, and the
        // second the display spends lighting up is a second the radio could
        // have spent delivering it.
        if let Some(link) = &mut link {
            link.send(address.read(), &reading);
        }
        panel.show(&reading);

        // Anything that arrived while the meter was busy is dropped here, so
        // the next `wait_command` starts from silence -- the same rule the
        // button already follows.
        if let Some(link) = &mut link {
            link.discard_pending().await;
        }
```

```
186
187         Timer::after_millis(DISPLAY_ON_MS).await;
188         panel.blank();
189     }
190 }
```

代码 2 src/meter.rs：一次测量的完整时序

```rust
//! One reading, start to finish: switch the external circuit in, bring the
//! analog blocks up, probe, range, take the frame, put everything back to
//! sleep, and turn the samples into amps.
//!
//! Everything true of a reading regardless of what asked for it lives here, so
//! the binaries differ only in *when* they measure and what they do with the
//! answer -- `main.rs` on a button press onto the panel, `bin/stream.rs` on a
//! timer out the radio. Neither can get the sequence subtly different from the
//! other, which matters because the order in `measure` is not arbitrary: the
//! calibration tables are only valid for readings taken exactly this way.

use embassy_mspm0::dma::Channel;
use embassy_mspm0::gpio::{Level, Output};
use embassy_mspm0::{Peri, peripherals};
use embassy_time::Timer;

use crate::cal::lsb_to_amps;
use crate::dsp::{coarse_pivot, rms_lsb};
use crate::range::{
    Gain, OUT_CENTER, Range, apply_range, dac_code_for, next_range, over_range, power_down_opa1,
    probe_stats, setup_opa1_pga,
};
use crate::sampler::{
    ADC_CH_RAW_IN, BUF, PINCM_PA16, PINCM_PA18, PROBE_BUF, capture, init_adc1_event, init_timer,
    set_adc_channel, set_hiz,
};

/// Settling allowed after the external circuit is switched into its measuring
/// state, before the probe frame starts.
///
/// What survives into the measurement frame is a *decaying* offset, and
/// `rms_lsb` subtracts the frame's own weighted mean -- that removes a constant
/// but not a decay, so the tail of the turn-on transient lands in the reading.
/// Set from the front end's measured turn-on time constant, long enough that
/// the residual at the start of the measurement frame is below a LSB.
const AFE_SETTLE_MS: u64 = 20;

/// How much a reading is worth. Ordered by which fault to report when more
/// than one is true, worst first -- that ordering is the reason this is one
/// enum rather than three flags each consumer re-prioritises for itself.
#[derive(Clone, Copy, PartialEq, Eq)]
pub enum Quality {
    /// The 2.5 V reference never reported ready, so every code in the frame is
    /// against an unsettled reference. Worst of the four because it is the only
    /// one where the samples look entirely ordinary: they are a plausible
    /// waveform, scaled by an unknown amount.
    RefBad,
    /// The probe found the input pinned at an ADC end, so the signal is
    /// leaving `0..VDDA` and no internal gain or bias choice can recover it --
    /// that needs external conditioning.
    InputBad,
    /// The arithmetic says the signal will not fit even on Direct, the least
    /// demanding range there is, so the frame is clipping. Every range is
    /// bounded by the supply and nothing tighter (see `range::OUT_LO` --
    /// OPA1's rails and ADC1's full scale are the same two voltages), which
    /// makes this exactly the same physical condition as `InputBad`, predicted
    /// from the probe's mean/pp rather than seen as pinned samples.
    OverRange,
    /// One of this cycle's DMA transfers never drained, so its buffer is part
    /// new samples and part stale ones.
    Incomplete,
    Good,
}

/// One measurement. `rms` and `range` together are one row of one CAL table,
/// which is why they travel with the answer rather than being consumed by it:
```

```rust
/// a reading that cannot say which table produced it cannot be calibrated
/// against, and one taken while `quality` is not `Good` must not be entered
/// into a table at all -- freezing that fiction into the instrument is worse
/// than having no row.
pub struct Reading {
    pub amps: f32,
    pub rms: f32,
    pub range: Range,
    /// Probe mean and peak-to-peak, in input-referred ADC LSB. `None` when the
    /// probe frame never drained, which is also when the range and the pivot
    /// are the previous cycle's rather than this one's.
    #[allow(dead_code)] // Read by bin/stream.rs, not by the panel.
    pub probe: Option<(i32, u32)>,
    ref_bad: bool,
    input_bad: bool,
    range_over: bool,
    incomplete: bool,
}

impl Reading {
    pub fn quality(&self) -> Quality {
        if self.ref_bad {
            Quality::RefBad
        } else if self.input_bad {
            Quality::InputBad
        } else if self.range_over {
            Quality::OverRange
        } else if self.incomplete {
            Quality::Incomplete
        } else {
            Quality::Good
        }
    }
}

pub struct Meter {
    afe_enable: Output<'static>,
    afe_enable_n: Output<'static>,
    /// Survive across cycles because the analog blocks do not: everything is
    /// powered down between readings, so each cycle rebuilds the front end
    /// from these two numbers rather than from whatever the registers held.
    ///
    /// `range` is where the next `next_range` starts from, and one cycle ago is
    /// a better guess than the ladder's pessimistic end. `mean_in_last` is the
    /// input DC the pivot is placed against.
    range: Range,
    mean_in_last: i32,
}

impl Meter {
    /// Claims the external circuit's control pair and puts the two analog pins
    /// in their idle state. Nothing else in this firmware touches PINCM40 or
    /// PINCM38.
    ///
    /// The initial range and pivot are the choice that assumes least -- x2
    /// cannot clip anything a higher step would have handled, and centre is the
    /// placement furthest from both rails. Neither survives the first cycle:
    /// the probe reads the input at unity gain and `next_range`/`dac_code_for`
    /// derive both from what it actually measured. Convergence takes one cycle
    /// rather than several, because the probe's numbers are input-referred and
    /// so do not depend on which range happens to be selected while it runs.
    pub fn new(
        pa8: Peri<'static, peripherals::PA8>,
        pa26: Peri<'static, peripherals::PA26>,
    ) -> Self {
        // The external circuit's two control lines, driven as one complementary
        // pair: measuring is PA8 high with PA26 low, idle is the reverse. They
        // are never set independently, which is why `set_measuring` takes a
        // single bool -- a state where both say "measure" or neither does is
```

```
        // not a state this circuit has, and the pair should not be able to
        // express one.
        //
        // Both pins need a physical resistor holding their idle level: between
        // power-on and this line they are inputs, not driven outputs, so PA8
        // needs a pull-down and PA26 a pull-up. That window is exactly when the
        // harvested supply has the least to spare, and it is also long -- it
        // covers the whole boot, not just these two statements.
        let meter = Self {
            afe_enable: Output::new(pa8, Level::Low),
            afe_enable_n: Output::new(pa26, Level::High),
            range: Range::Pga(Gain::X2),
            mean_in_last: OUT_CENTER,
        };
        set_hiz(PINCM_PA18);
        set_hiz(PINCM_PA16);
        meter
    }

    /// Leaving is the mirror of entering, so the circuit passes through the
    /// same intermediate state in both directions rather than a different one
    /// each way.
    fn set_measuring(&mut self, measuring: bool) {
        if measuring {
            self.afe_enable.set_high();
            self.afe_enable_n.set_low();
        } else {
            self.afe_enable_n.set_high();
            self.afe_enable.set_low();
        }
    }

    /// One complete cycle. Returns with every analog block powered down and the
    /// external circuit back in its idle state, whatever happened in between.
    pub async fn measure(&mut self, dma: &mut Channel<'_>) -> Reading {
        self.set_measuring(true);

        // The analog blocks come up inside the front end's own settling window,
        // so bringing them back costs no latency of its own: the DAC's turn-on
        // (datasheet 7.17.5, 6.9 us max) and OPA1's enable time (7.19.2, 6 us
        // max at GBW=HIGHGAIN) are microseconds against AFE_SETTLE_MS here.
        // Each is a full bring-up, not a resume: the end of every cycle clears
        // PWREN on all four blocks, and that resets every register in them.
        //
        // The gain and the pivot are last cycle's, so a cycle whose probe fails
        // still measures at a setting that was right rather than at a blind
        // mid-scale one. The probe replaces both a few milliseconds from now.
        //
        // The reference goes first and everything downstream is measured
        // against it: the DAC's bias, the ADC's full scale, and therefore both
        // the window the autoranger fits into and the codes the RMS is formed
        // from. Its 200 us settle (datasheet 7.15.2) hides inside AFE_SETTLE_MS
        // like the rest.
        let ref_bad = !crate::vref::init(crate::vref::Output::V2_5);
        let gain = match self.range {
            Range::Pga(g) => g,
            Range::Direct => Gain::X2,
        };
        setup_opa1_pga(gain, dac_code_for(gain, self.mean_in_last));
        init_adc1_event();
        init_timer();

        Timer::after_millis(AFE_SETTLE_MS).await;

        // Both reset every cycle: one cycle is one independent reading, so a
        // mark can only ever describe the frames this cycle actually took.
        let mut input_bad = false;
        let mut range_over = false;

```

```
        // --- Probe: the raw input at unity gain, straight off PA18 ---
        //
        // The OPA is not involved and does not move, so this costs one MEMCTL
        // write and 40 ms, with no settle. Both numbers it yields are
        // input-referred and therefore independent of the gain currently
        // selected, which is what lets the range converge in a single cycle
        // rather than hunting toward it.
        set_adc_channel(ADC_CH_RAW_IN);
        // SAFETY: `capture` waits the transfer out (or pauses it) before
        // returning, so nothing else touches PROBE_BUF concurrently.
        let probed = capture(dma, unsafe { &mut *core::ptr::addr_of_mut!(PROBE_BUF
↪ ) }).await;
        let mut probe = None;
        if probed {
            // SAFETY: as above -- the transfer is finished or paused.
            let (mean_in, pp_in, railed) = probe_stats(unsafe { &*core::ptr::
↪ addr_of!(PROBE_BUF) });
            probe = Some((mean_in, pp_in));
            input_bad = railed;
            if !railed {
                self.mean_in_last = mean_in;
                self.range = next_range(self.range, mean_in, pp_in);
                range_over = over_range(self.range, mean_in, pp_in);
                if apply_range(self.range, mean_in) {
                    // Settle the DAC and the re-tapped ladder before the frame
                    // that carries the marks. Both are microsecond-scale; 1 ms
                    // is slack.
                    // Skipped on Direct, which changes nothing analog.
                    Timer::after_millis(1).await;
                }
            }
        }
        // A failed or railed probe leaves the range exactly as it was.
        // Re-ranging on numbers known to be wrong is strictly worse than
        // keeping a setting that was right one cycle ago.

        // --- Measurement frame: whatever the chosen range put the signal on
        // (OPA1_OUT on PA16 for the PGA ranges, PA18 itself for Direct, in
        // which case the channel is already the one the probe just used) ---
        // SAFETY: `capture` waits the transfer out (or pauses it) before
        // returning, so nothing else touches BUF concurrently.
        let framed = capture(dma, unsafe { &mut *core::ptr::addr_of_mut!(BUF) }).
↪ await;

        // Everything below this line works out of BUF, so the whole signal
        // chain comes down here rather than after the caller is done with the
        // answer: the front end has no reader left, and the reader has nothing
        // left to read.
        //
        // Order is downstream first. The converter and its sample clock stop,
        // then the amplifier and its bias reference, then the external circuit
        // -- so nothing is ever driving into a block that has just lost power,
        // and the 2.5 V reference outlives both of the blocks that use it.
        // What this leaves on PA18 is the pad alone: the IOMUX has held it
        // high-Z since boot, and with OPA1 and ADC1 out of the power domain
        // there is no longer an analog mux on the other side of it either.
        crate::sampler::power_down();
        power_down_opa1();
        self.set_measuring(false);

        // BUF holds 800 hardware-timed samples at 4 kHz (TIM+ADC+DMA, zero CPU
        // involvement per sample).
        // SAFETY: as above -- the transfer is finished or paused, so BUF is
        // ours until the next `measure`.
        let buf = unsafe { &*core::ptr::addr_of!(BUF) };
        let pivot = coarse_pivot(buf);
        let rms = rms_lsb(buf, pivot);

        Reading {
```

```
271             amps: lsb_to_amps(rms, self.range),
272             rms,
273             range: self.range,
274             probe,
275             ref_bad,
276             input_bad,
277             range_over,
278             incomplete: !probed || !framed,
279         }
280     }
281 }
```

代码 3 src/dsp.rs：加窗、真有效值与工频估计

```rust
//! Everything that turns one captured frame into numbers: the window and
//! quadrature tables, true RMS, the mains-frequency estimate, and the
//! frame-to-frame spread indicator. Pure arithmetic over a sample buffer --
//! no registers, no knowledge of which range produced the samples.

use libm::{atan2f, sqrtf};

use crate::sampler::{ADC_MASK, N, SAMPLE_HZ};

const TWO_PI: f32 = 2.0 * core::f32::consts::PI;
const PI: f32 = core::f32::consts::PI;

/// Nominal mains frequency. Only the *correlator* in `estimate_hz` assumes
/// this value; the RMS path (`rms_lsb`) does not depend on the mains
/// frequency at all -- see the Hann note there.
const MAINS_NOM_HZ: u32 = 50;

/// Samples per nominal mains cycle -- 80. Must divide evenly, so the
/// quadrature tables can be indexed by `n % SPP_NOM` instead of carrying a
/// running phase (no drift, no per-sample sin/cos on a Cortex-M0+ that has
/// no FPU).
const SPP_NOM: usize = (SAMPLE_HZ / MAINS_NOM_HZ) as usize;

/// Half-record length for the two-block phase-difference frequency
/// estimate.
const HALF: usize = N / 2;

const _: () = assert!(
    SAMPLE_HZ.is_multiple_of(MAINS_NOM_HZ),
    "SPP_NOM must be exact"
);

const _: () = assert!(N.is_multiple_of(2));

// The whole phase-difference trick rests on the *nominal* fundamental
// advancing an exact multiple of 2*pi across HALF samples (400 * 2pi/80 =
// 10pi). If HALF stopped being a whole number of nominal cycles, the wrapped
// phase difference would carry a constant bias instead of reading out pure
// frequency deviation.
const _: () = assert!(HALF.is_multiple_of(SPP_NOM));

/// Cosine for const evaluation, argument in radians.
///
/// Exists because `libm::cosf` is not a `const fn`, and the tables below are
/// worth having as compile-time constants: see the note on `HANN`.
///
/// Quadrant-reduced to [0, pi/2] and then a Taylor series to x^14. At the
/// reduced argument's worst case (pi/2) the first dropped term is ~1.6e-9,
/// well under an f32 ULP, and the whole table checks out at 1.4e-7 max
/// absolute error against a f64 reference -- about 1.2 ULP at magnitude 1.
/// A window function needs nothing like that precision (the Hann sidelobe
/// argument in `rms_lsb` would survive 1e-5), so this has margin to spare.
const fn cos_const(x: f32) -> f32 {
    let mut t = x;
    while t < 0.0 {
        t += TWO_PI;
    }
    while t >= TWO_PI {
        t -= TWO_PI;
    }
    // cos(2pi - t) = cos(t), so fold onto [0, pi].
    if t > PI {
        t = TWO_PI - t;
    }
    // cos(pi - t) = -cos(t), so fold onto [0, pi/2] and carry the sign.
    let sign = if t > PI / 2.0 {
        t = PI - t;
        -1.0
```

```
    } else {
        1.0
    };

    let x2 = t * t;
    let mut term = 1.0f32;
    let mut sum = 1.0f32;
    let mut k = 1usize;
    while k <= 7 {
        term = -term * x2 / ((2 * k - 1) * (2 * k)) as f32;
        sum += term;
        k += 1;
    }
    sign * sum
}

/// Sine for const evaluation. Same reduction shape as `cos_const`, with its
/// own series rather than `cos(pi/2 - x)` -- the subtraction would move the
/// rounding error into the argument, where it is worth the most for small x.
#[allow(dead_code)] // Retained for the optional bench frequency estimator.
const fn sin_const(x: f32) -> f32 {
    let mut t = x;
    while t < 0.0 {
        t += TWO_PI;
    }
    while t >= TWO_PI {
        t -= TWO_PI;
    }
    // sin(t + pi) = -sin(t), then sin(pi - t) = sin(t).
    let mut sign = 1.0f32;
    if t > PI {
        t -= PI;
        sign = -1.0;
    }
    if t > PI / 2.0 {
        t = PI - t;
    }

    let x2 = t * t;
    let mut term = t;
    let mut sum = t;
    let mut k = 1usize;
    while k <= 7 {
        term = -term * x2 / ((2 * k) * (2 * k + 1)) as f32;
        sum += term;
        k += 1;
    }
    sign * sum
}

/// Periodic (DFT-even) Hann, `w[n] = 0.5*(1 - cos(2*pi*n/N))`. The periodic
/// form is the one that makes `sum(w)` come out to exactly N/2 and puts the
/// window's nulls on the analysis bins, which is the whole point here.
///
/// Computed at compile time, so it costs flash instead of SRAM and nothing
/// at startup. It used to be built by 401 runtime `cosf` calls (~19 ms on a
/// soft-float M0+) into a `static mut`; startup time is a scored item, and
/// that was the single largest avoidable chunk of it outside the HC-42's own
/// 500 ms settle. Being a plain `static` also retires the last `static mut`
/// in this module along with the `unsafe` and the `Tables` handle that
/// existed only to contain it.
///
/// Still mirrored about n = N/2. At const-eval time that is no longer about
/// speed: it makes `HANN[n] == HANN[N - n]` hold bit-for-bit, so the window
/// is *exactly* symmetric and cannot bias the phase estimate in `half_phase`.
static HANN: [f32; N] = {
    let mut w = [0.0f32; N];
    let mut n = 0;
    while n <= N / 2 {
```

```
        let v = 0.5 * (1.0 - cos_const(TWO_PI * n as f32 / N as f32));
        w[n] = v;
        if n > 0 && n < N / 2 {
            w[N - n] = v;
        }
        n += 1;
    }
    w
};

/// One nominal-frequency cycle of quadrature, for the `estimate_hz`
/// correlator. Same story as `HANN`: compile-time, flash-resident.
#[allow(dead_code)] // Retained for the optional bench frequency estimator.
static COS_TAB: [f32; SPP_NOM] = {
    let mut t = [0.0f32; SPP_NOM];
    let mut n = 0;
    while n < SPP_NOM {
        t[n] = cos_const(TWO_PI * n as f32 / SPP_NOM as f32);
        n += 1;
    }
    t
};

#[allow(dead_code)] // Retained for the optional bench frequency estimator.
static SIN_TAB: [f32; SPP_NOM] = {
    let mut t = [0.0f32; SPP_NOM];
    let mut n = 0;
    while n < SPP_NOM {
        t[n] = sin_const(TWO_PI * n as f32 / SPP_NOM as f32);
        n += 1;
    }
    t
};

/// How many recent frames the spread indicator remembers.
#[allow(dead_code)] // Retained for bench noise characterization.
pub const SPREAD_N: usize = 16;

/// Peak-to-peak spread of the last SPREAD_N raw frame RMS values, in LSB.
///
/// Deliberately *displayed rather than filtered*. Cross-frame averaging was
/// tried here and removed: any IIR/running mean carries hidden state whose
/// lag is invisible on the display, and it misbehaves exactly in the band
/// that matters. A step smaller than the reset threshold but larger than the
/// 0.5% spec (say a 1.00 -> 1.01 A load nudge) is not detected as a change,
/// so the reading creeps toward it over tens of frames while the settle
/// indicator claims to be settled. A bench DMM has an aperture (NPLC) and
/// emits one reading per aperture; it keeps no memory across readings. If
/// this board turns out to need less noise, the correct lever is a longer
/// aperture -- accumulate M consecutive captures into one reading, which is
/// bounded, has no lag beyond the aperture itself, and stays fully valid one
/// aperture after any load change.
///
/// What this gives instead: the frame-to-frame spread, on the actual bench,
/// with no noise model assumed. That is the sigma measurement that decides
/// whether a longer aperture is needed at all -- and it comes for free from
/// the display, with no separate procedure.
#[allow(dead_code)] // Retained for bench noise characterization.
pub struct Spread {
    hist: [f32; SPREAD_N],
    idx: usize,
    len: usize,
}

#[allow(dead_code)] // Retained for bench noise characterization.
impl Spread {
    pub const fn new() -> Self {
        Self {
            hist: [0.0; SPREAD_N],
```

```
            idx: 0,
            len: 0,
        }
    }

    /// Records one frame and returns the current peak-to-peak spread.
    pub fn push(&mut self, x: f32) -> f32 {
        self.hist[self.idx] = x;
        self.idx = (self.idx + 1) % SPREAD_N;
        if self.len < SPREAD_N {
            self.len += 1;
        }

        let mut lo = self.hist[0];
        let mut hi = self.hist[0];
        for &v in &self.hist[..self.len] {
            if v < lo {
                lo = v;
            }
            if v > hi {
                hi = v;
            }
        }
        hi - lo
    }
}

// NOTE ON THE NOISE BIAS -- deliberately *not* corrected here.
//
// Noise adds to the signal in quadrature, so the raw estimate reads
// sqrt(signal^2 + sigma^2), high by about sigma^2/(2*rms^2). It is a
// one-sided bias rather than spread, so frame averaging and any smoothing
// filter leave it untouched. At 0.1 A (rms ~29.1 LSB) it is worth 0.06% for
// sigma = 1 LSB and 0.53% for sigma = 3 LSB; at 2 A it is 400x smaller.
//
// It must NOT be corrected with a hardcoded sigma^2. The harvesting front
// end has least available power at light load, so its ripple is worst
// exactly where the spec is tightest -- sigma is a function of load current,
// not a constant. Simulating sigma rising 1 LSB (at 2 A) to 4 LSB (at 0.1 A):
// a two-point k/b fit alone leaves 0.23% worst-case, while subtracting a
// fixed sigma^2 = 9 leaves 0.41%. A constant correction is worse than none.
//
// What CAL_OFFSET does instead: a two-point fit absorbs most of this bias,
// because b lands near -sigma^2/(2*rms_lo) by construction. Residual is a
// bowl, zero at both fit points, worst near 0.2 A -- 0.12% at sigma = 3 LSB,
// 0.33% at 5, 1.26% at 10. So the bench measurement to make first is sigma
// *at several load currents*: if it stays under ~3 LSB across the range,
// CAL_OFFSET alone provably covers it and nothing more is needed. If it does
// not, the fix is to estimate the noise per frame at runtime (its power is
// separable -- all real load current sits on harmonics of the line
// frequency, so inter-harmonic bins such as 75/125/175 Hz see noise only),
// never to freeze a number into flash.
//
// To measure sigma: open the load, leave everything else powered exactly as
// in normal operation (the rectifier and supercap charging are the suspected
// source, so they must be live), and read the raw `rms` line off the OLED --
// that reading *is* sigma in LSB. Repeat under load at several currents by
// comparing the reading's spread against the reference ammeter.

/// Coarse DC pivot, integer and exact-to-1-LSB. Deliberately *not* the DC
/// estimate: it only has to land within a few LSB of the ~2048-count OPA
/// bias so the f32 accumulators downstream never hold a large sum whose
/// small difference matters. Its truncation error is solved out exactly by
/// the weighted offset in `rms_lsb`, which is why the old integer-division
/// mean (up to 1 LSB off, worth a one-sided ~0.06% RMS bias at 0.1 A) is
/// no longer a problem.
pub fn coarse_pivot(buf: &[u32; N]) -> i32 {
    (buf.iter().map(|&x| (x & ADC_MASK) as u64).sum::<u64>() / N as u64) as i32
}
```

```
276
277 /// Hann-weighted true RMS over one captured frame, in raw ADC-LSB units.
278 ///
279 /// Why weighted: a plain rectangular average of x^2 is only unbiased when
280 /// the window spans a whole number of mains cycles. The error term is the
281 /// *weight* sequence's frequency response evaluated at 2f -- bin 20 of this
282 /// 200 ms record. A rectangular window's sidelobes there are only ~-35 dB,
283 /// which at 49.66 Hz is a 0.33% RMS error; Hann's are below -95 dB, so the
284 /// term vanishes. That happens without knowing the mains frequency at all,
285 /// which also makes it immune to SAMPLE_HZ itself being wrong (SYSOSC is an
286 /// internal RC, ~+-1%, and only the ratio f/SAMPLE_HZ ever mattered).
287 /// Harmonic h lands at bin 20h, i.e. suppressed even harder, so distorted
288 /// load currents are covered by the same mechanism -- and no zero crossing
289 /// is needed, so a dead-zone waveform is fine too.
290 ///
291 /// Cost: effective noise bandwidth x1.5, taking the per-frame noise floor
292 /// from ~0.035% to ~0.043% at 0.1 A. Irrelevant against a 0.5% budget.
293 ///
294 /// TESTING TRAP: do not bench this with a signal generator set to exactly
295 /// 50.000 Hz. At exactly SAMPLE_HZ/SPP_NOM the record contains only 80
296 /// distinct signal phases repeated 10 times, so the quantization error is
297 /// perfectly correlated with the signal and does not average down at all --
298 /// simulation puts the worst-phase error at 0.51% at 0.1 A, and no window
299 /// can fix it because it is not leakage. Real mains never sits exactly on
300 /// 50.000 Hz (and SYSOSC guarantees SAMPLE_HZ does not either), so the
301 /// incoherent case is the real one; at 49.8 Hz the same test gives 0.07%.
302 /// A generator at 50.02 Hz reproduces field behaviour, 50.000 does not.
303 pub fn rms_lsb(buf: &[u32; N], pivot: i32) -> f32 {
304     // Pass 1: weighted mean of the pivot-centred residual. The mean has to
305     // be weighted by the *same* window as the sum of squares -- an
306     // unweighted mean leaves fundamental residue that Hann cannot then
307     // suppress, and re-introduces the error this function exists to remove.
308     let mut sum_w = 0.0f32;
309     let mut sum_wd = 0.0f32;
310     for (&x, &w) in buf.iter().zip(HANN.iter()) {
311         sum_w += w;
312         sum_wd += w * ((x & ADC_MASK) as i32 - pivot) as f32;
313     }
314     // True weighted DC, expressed as an offset from the integer pivot.
315     let offset = sum_wd / sum_w;
316
317     // Pass 2. Two passes are mandatory, not stylistic: folding this into
318     // pass 1 as sum(w*x^2)/sum(w) - mean^2 subtracts two ~4.2e6 quantities
319     // to recover ~848, and f32 has no mantissa left for that cancellation.
320     let mut sum_wd2 = 0.0f32;
321     for (&x, &w) in buf.iter().zip(HANN.iter()) {
322         let d = ((x & ADC_MASK) as i32 - pivot) as f32 - offset;
323         sum_wd2 += w * d * d;
324     }
325
326     sqrtf(sum_wd2 / sum_w)
327 }
328
329 /// Phase of the MAINS_NOM_HZ component of one HALF-sample block, measured
330 /// at the block's midpoint, via direct quadrature correlation.
331 ///
332 /// For x[n] = A*sin(w*n + phi) correlated against the nominal w0:
333 ///   i_acc ~ (A/2)*sum(w)*sin(phi),  q_acc ~ (A/2)*sum(w)*cos(phi)
334 /// hence `atan2(i, q)`. Hann-weighted first: the image term at 2*f0 sits
335 /// only 10 bins away in a 400-sample block, where a rectangular window
336 /// leaks ~3% and would swing the phase by up to ~0.03 rad -- about 0.05 Hz.
337 /// A 400-point Hann is exactly the 800-point table decimated by 2.
338 ///
339 /// `start` indexes the record, but the correlator is driven by the *local*
340 /// index, which is what makes the two blocks' phase difference come out as
341 /// the fundamental's advance across HALF samples.
342 #[allow(dead_code)] // Retained for the optional bench frequency estimator.
343 pub fn half_phase(buf: &[u32; N], start: usize, pivot: i32) -> f32 {
344     let mut i_acc = 0.0f32;
```

```
    let mut q_acc = 0.0f32;
    for m in 0..HALF {
        let d = ((buf[start + m] & ADC_MASK) as i32 - pivot) as f32 * HANN[2 * m];
        let p = m % SPP_NOM;
        i_acc += d * COS_TAB[p];
        q_acc += d * SIN_TAB[p];
    }
    atan2f(i_acc, q_acc)
}

/// Mains frequency from the phase the fundamental advances between the two
/// halves of one record.
///
/// Across HALF samples the *nominal* fundamental advances 400*2pi/80 = 10pi
/// -- an exact multiple of 2pi -- so the wrapped phase difference is purely
/// the deviation:
///
///   dphi = 2*pi * (f - 50) * HALF / SAMPLE_HZ = 0.6283 * (f - 50)
///
/// Unambiguous over +-5 Hz, far wider than the grid ever wanders. This uses
/// all 800 samples with SNR-optimal weighting instead of the ~18 samples
/// nearest zero, so it neither cares about noise at the zero crossing nor
/// can be fooled by a spurious extra crossing.
///
/// Reported for the display and the design report only -- `rms_lsb` does
/// not consume it, so a bad frequency estimate cannot corrupt the accuracy
/// figure that carries the marks.
#[allow(dead_code)] // Retained for the design report and bench diagnostics.
pub fn estimate_hz(buf: &[u32; N], pivot: i32) -> f32 {
    let p1 = half_phase(buf, 0, pivot);
    let p2 = half_phase(buf, HALF, pivot);

    let mut dphi = p2 - p1;
    while dphi > core::f32::consts::PI {
        dphi -= TWO_PI;
    }
    while dphi <= -core::f32::consts::PI {
        dphi += TWO_PI;
    }

    let phi_per_hz = TWO_PI * HALF as f32 / SAMPLE_HZ as f32;
    MAINS_NOM_HZ as f32 + dphi / phi_per_hz
}
```

代码 4 src/link.rs：地址、帧格式与收发

```rust
//! The wireless link: HC-42 on UART2, the address switch that names this
//! meter, the frames that go out and the one command that comes in.
//!
//! Shared by both binaries so there is exactly one wire format. `main.rs`
//! keeps the link up and measures when told to; `bin/stream.rs` keeps it up and
//! measures on a timer. Neither can drift from the other's grammar.

use core::fmt::Write as _;

use embassy_mspm0::Peri;
use embassy_mspm0::gpio::{Input, Level, Output, Pull};
use embassy_mspm0::interrupt::typelevel::Binding;
use embassy_mspm0::peripherals;
use embassy_mspm0::uart::{BufferedInterruptHandler, BufferedUart, Config as
↪ UartConfig, Instance};
use embassy_time::Timer;
use embedded_io_async::{Read, ReadReady};

use crate::meter::{Quality, Reading};

/// HC-42 settling after its supply comes up, before UART2 exists. The module
/// must not see the idle-high TX line while unpowered -- current would flow
/// back through its RXD protection diode -- so this delay is also what
/// separates "supply up" from "pin driven".
const BT_SETTLE_MS: u64 = 350;

/// Not the HC-42's factory 9600: at that rate one reading's two frames are 110
/// bytes and 115 ms of pure wire time, which lands directly in how long a
/// reader waits for an answer. 115200 makes the same frames 9.6 ms and costs
/// nothing else -- the module supports up to 230400 (HC42.pdf 5.3.5).
///
/// **The module must be set to match**, with `AT+UART=115200`; `bin/btcfg.rs`
/// does it and reads the value back. A module left at 9600 is not slow, it is
/// silent, which is the failure this constant exists to make visible: change it
/// here and the meter goes mute until the module is changed too.
pub const BT_BAUD_RATE: u32 = 115_200;

/// Alarm thresholds from 2(3), in mA. The phone classifies again from its own
/// preferences; sending our own classification means a meter read by anything
/// else still says what it thinks.
const LOW_LIMIT_MA: u32 = 200;

const HIGH_LIMIT_MA: u32 = 2_000;

/// `METER_TEST` allows 1-7 digits. A reading that would overflow that is
/// nonsense anyway, and a malformed frame is worse than a saturated one: the
/// parser drops it whole, so the fault would look like silence.
const MAX_FRAME_MA: u32 = 9_999_999;

/// Longest command line accepted. Anything longer is a reader that has lost
/// the plot or line noise that happens to lack newlines; either way the buffer
/// resets rather than growing.
const CMD_MAX: usize = 32;

/// How often the meter announces itself while idle. Cheap enough to ignore:
/// one 17-byte line is 18 ms of 9600-baud UART and no analog blocks at all,
/// against the 1.22 mA the HC-42 already draws holding a full-speed connection
/// (HC42.pdf 1.3).
///
/// It is what makes the reader's view of "who is out there" track the coded
/// switch: one meter stands in for 16 addresses, and when the switch moves, the
/// old address simply stops announcing and the new one starts. It also makes a
/// disconnected load visible within a few seconds -- the meter loses power with
/// it, so the announcements just stop -- which is exactly the offline condition
/// of 2(3), and far quicker than waiting for a poll to time out.
pub const HEARTBEAT_MS: u64 = 2_000;

/// What a reader can ask for. One variant today, and the parser is written so
```

```rust
/// that stays cheap to extend.
#[derive(Clone, Copy, PartialEq, Eq)]
pub enum Command {
    /// Take one reading now. Equivalent to a press of S2, including what
    /// happens to a second one that arrives mid-measurement: nothing.
    Measure,
}

/// The four-position coded switch, ON to ground against internal pull-ups, so
/// a closed switch reads low and contributes its weight.
///
/// Read per frame rather than latched at boot: the address is meant to be set
/// on the bench with the meter running, and a value that only takes effect
/// after a power cycle is a value that gets set wrong once and stays wrong.
pub struct Address {
    bit0: Input<'static>,
    bit1: Input<'static>,
    bit2: Input<'static>,
    bit3: Input<'static>,
}

impl Address {
    pub fn new(
        pb0: Peri<'static, peripherals::PB0>,
        pb6: Peri<'static, peripherals::PB6>,
        pb7: Peri<'static, peripherals::PB7>,
        pb8: Peri<'static, peripherals::PB8>,
    ) -> Self {
        Self {
            bit0: Input::new(pb0, Pull::Up),
            bit1: Input::new(pb6, Pull::Up),
            bit2: Input::new(pb7, Pull::Up),
            bit3: Input::new(pb8, Pull::Up),
        }
    }

    pub fn read(&self) -> u8 {
        (self.bit0.is_low() as u8)
            | (self.bit1.is_low() as u8) << 1
            | (self.bit2.is_low() as u8) << 2
            | (self.bit3.is_low() as u8) << 3
    }
}

pub struct Link {
    uart: BufferedUart<'static>,
    /// Partial command line, carried across reads because a 9600-baud line
    /// arrives a few bytes at a time.
    pending: heapless::String<CMD_MAX>,
    _power: Output<'static>,
}

impl Link {
    /// Power the radio and open the port, in that order and with the settle in
    /// between (board notes 6.1). Stays up from here: a module that has lost
    /// power cannot be woken over the air, so the only thing that could turn it
    /// back on is the local button -- which defeats the point of accepting
    /// commands at all.
    ///
    /// PB17 needs an external pull-down for the window before this runs, or the
    /// radio powers up during flashing and reset.
    pub async fn power_up<T: Instance>(
        pb17: Peri<'static, peripherals::PB17>,
        uart: Peri<'static, T>,
        tx: Peri<'static, impl embassy_mspm0::uart::TxPin<T>>,
        rx: Peri<'static, impl embassy_mspm0::uart::RxPin<T>>,
        irq: impl Binding<T::Interrupt, BufferedInterruptHandler<T>>,
        tx_buf: &'static mut [u8],
        rx_buf: &'static mut [u8],
```

```
    ) -> Option<Self> {
        let power = Output::new(pb17, Level::High);
        Timer::after_millis(BT_SETTLE_MS).await;

        let mut config = UartConfig::default();
        config.baudrate = BT_BAUD_RATE;
        let uart = BufferedUart::new(uart, tx, rx, irq, tx_buf, rx_buf, config).ok
↪ ()?;
        Some(Self {
            uart,
            pending: heapless::String::new(),
            _power: power,
        })
    }

    /// Wait for a command. Bytes that arrive while nobody is waiting are held
    /// by the buffered UART, so a command sent a moment early is not lost --
    /// only one sent *during* a measurement is, and that is deliberate: see
    /// `discard_pending`.
    pub async fn wait_command(&mut self, addr: u8) -> Command {
        let mut byte = [0u8; 1];
        loop {
            if self.uart.read(&mut byte).await.is_err() {
                continue;
            }
            if let Some(command) = self.feed(byte[0], addr) {
                return command;
            }
        }
    }

    /// Throw away everything buffered, partial line included.
    ///
    /// Called after a measurement so a command that arrived during one is
    /// dropped rather than queued -- the same rule the button follows, where
    /// `wait_for_falling_edge` clears pending edges before it arms. A reading
    /// answers the request that started it; a request made while the meter was
    /// busy would otherwise be answered by a reading it did not ask for, and
    /// the reader has no way to tell the two apart.
    pub async fn discard_pending(&mut self) {
        self.pending.clear();
        let mut sink = [0u8; 32];
        // `read_ready` is what keeps the await from parking: it only runs when
        // there is something buffered, so this drains and returns rather than
        // waiting for a byte that may never come.
        while self.uart.read_ready().unwrap_or(false) {
            if self.uart.read(&mut sink).await.is_err() {
                break;
            }
        }
    }

    /// One byte into the line assembler. Returns a command on the newline that
    /// completes one.
    fn feed(&mut self, byte: u8, addr: u8) -> Option<Command> {
        if byte == b'\n' || byte == b'\r' {
            let command = parse_command_for(self.pending.trim(), addr);
            self.pending.clear();
            return command;
        }
        // A line that outgrows the buffer is dropped rather than truncated:
        // truncation can turn an unknown command into a known prefix of one.
        if self.pending.push(byte as char).is_err() {
            self.pending.clear();
        }
        None
    }

    /// "I am here, and I am meter n." Carries no reading on purpose: this is a
```

```
    /// presence beat, not a measurement, and nothing analog wakes up for it.
    pub fn send_alive(&mut self, addr: u8) {
        let _ = writeln!(self, "IMHERE,ADDR={}", addr);
    }

    /// One reading, on the wire.
    ///
    /// `METER_TEST` is withheld when the reading is one the meter cannot stand
    /// behind, because that frame has nowhere to say so -- its STATUS field is
    /// the alarm classification, and sending a fiction as NORMAL is worse than
    /// sending nothing: a reader that stops hearing from a meter shows it
    /// offline, which is true, while a confident wrong number is not.
    ///
    /// `OverRange` is the exception. It means the signal will not fit even at
    /// unity gain, which on this front end is a current past the top of scale
    /// -- the number is understated but the direction is known, and that is
    /// exactly the case 2(3) wants alarmed. It goes out as HIGH.
    ///
    /// `METER_CAL` always goes out, including for the withheld cases. A frame
    /// the bench cannot use is still evidence, and it carries its own FLAG.
    pub fn send(&mut self, addr: u8, reading: &Reading) {
        let quality = reading.quality();
        let ma = milliamps(reading);

        let test_status = match quality {
            Quality::Good => Some(status(ma)),
            Quality::OverRange => Some("HIGH"),
            Quality::RefBad | Quality::InputBad | Quality::Incomplete => None,
        };
        if let Some(test_status) = test_status {
            let _ = writeln!(
                self,
                "METER_TEST,ADDR={},CURRENT_MA={},STATUS={}",
                addr, ma, test_status
            );
        }

        let _ = write!(
            self,
            "METER_CAL,ADDR={},RMS={:.2},GAIN={},FLAG={}",
            addr,
            reading.rms,
            reading.range.nominal(),
            flag(quality)
        );
        // Omitted rather than zeroed when the probe frame never drained: a mean
        // of 0 is a legal reading, and a bench that cannot tell it from "no
        // probe" would chase the wrong fault.
        if let Some((mean_in, pp_in)) = reading.probe {
            let _ = write!(self, ",MEAN={},PP={}", mean_in, pp_in);
        }
        let _ = writeln!(self);
    }
}

/// `core::fmt` sink, so frames are written with plain `write!` instead of
/// formatting into a `heapless::String` a line at a time.
impl Link {
    /// Push every byte, however many calls that takes.
    ///
    /// `BufferedUartTx::blocking_write` is a *short* write: it copies into the
    /// ring buffer's contiguous region and returns how much fit, which is less
    /// than asked for whenever the ring wraps mid-frame. Ignoring that count
    /// truncates frames at a boundary that moves with traffic, and a truncated
    /// frame does not read as a fault -- the parser simply drops it, so the
    /// meter looks intermittently silent instead of broken.
    fn put(&mut self, mut bytes: &[u8]) {
        while !bytes.is_empty() {
            match self.uart.blocking_write(bytes) {
```

```
                Ok(0) | Err(_) => return,
                Ok(n) => bytes = &bytes[n..],
            }
        }
    }
}

/// `core::fmt` sink, so frames are written with plain `write!` instead of
/// formatting into a `heapless::String` a line at a time.
impl core::fmt::Write for Link {
    /// Translates bare LF to CRLF. `MeterFrameParser` strips the CR itself, but
    /// every serial terminal used to look at this link mid-bench does not.
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        for part in s.split_inclusive('\n') {
            match part.strip_suffix('\n') {
                Some(head) => {
                    self.put(head.as_bytes());
                    self.put(b"\r\n");
                }
                None => self.put(part.as_bytes()),
            }
        }
        Ok(())
    }
}

/// `MEAS` alone is a broadcast; `MEAS,ADDR=n` is ignored unless `n` is this
/// meter's switch setting, so a reader can talk to one of several in range.
///
/// Deliberately not `METER_TEST`-shaped: commands travel the opposite
/// direction on the same transparent link, and a grammar a meter could mistake
/// for its own output is one echo away from a meter triggering itself.
pub fn parse_command_for(line: &str, addr: u8) -> Option<Command> {
    let command = parse_command(line)?;
    match line.split_once(",ADDR=") {
        None => Some(command),
        Some((_, want)) => (want.trim().parse::<u8>().ok()? == addr).then_some(
↪ command),
    }
}

fn parse_command(line: &str) -> Option<Command> {
    let head = line.split(',').next()?;
    match head {
        "MEAS" => Some(Command::Measure),
        _ => None,
    }
}

fn milliamps(reading: &Reading) -> u32 {
    // f32 -> u32 saturates at both ends in Rust, so a negative RMS (which the
    // CAL extrapolation can produce below the table's first point) lands on 0
    // rather than wrapping to something enormous.
    ((reading.amps * 1000.0) as u32).min(MAX_FRAME_MA)
}

fn status(milliamps: u32) -> &'static str {
    if milliamps < LOW_LIMIT_MA {
        "LOW"
    } else if milliamps > HIGH_LIMIT_MA {
        "HIGH"
    } else {
        "NORMAL"
    }
}

fn flag(quality: Quality) -> &'static str {
    match quality {
        Quality::RefBad => "REF_BAD",
```

```
342             Quality::InputBad => "BAD_INPUT",
343             Quality::OverRange => "OVER",
344             Quality::Incomplete => "PARTIAL",
345             Quality::Good => "OK",
346         }
347 }
```